Experiment No. 2

Title:

Advanced Class Concepts – Decorators and Magic methods

Aim:

Design a dynamic report generator in Python that uses decorators, class methods, and magic methods to customize and format reports. The system should allow users to define report templates and apply various formatting options dynamically.

Theory:

- **Decorators**: Functions that modify the behavior of other functions or methods. They are used to add functionality dynamically.
- **Class Methods**: Methods that are bound to the class and not the instance. They can modify the class state that applies across all instances.
- **Magic Methods**: Special methods in Python (e.g., __str__, __repr__, __call__) that enable customization of object behavior and overloading operators.

Decorators

Often overlooked because of their perceived complexity, Decorators are instead a very common and convenient design pattern that can enhance code quality, readability, and maintainability.

A decorator can be a class or function that influence or modify the behavior of another class or function without changing its source code.

1. Function Decorators

To create a decorator, as the name suggests, we need to decorate something else, in particular, in the example we are going to explore, we will decorate a function.
First of all, we need to create a function (**the decorator**), it has to take as an argument the function we want to modify (**decorate**) and wrap it around a **wrapper** function, that will contain the new logic we want to apply.

```
def log_function_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling function: {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Function {func.__name__} executed successfully.")
        return result
    return wrapper

@log_function_call
def add_numbers(a, b):
    return a + b

result = add_numbers(2, 3)
print(result)
```

• How it works

In the example above, we defined a decorator named log_function_call. It takes a function as an argument and returns a wrapper function. The wrapper function adds logging statements before and after the original function is executed. We, then, apply the decorator to the add_numbers function using the special Python syntax @. When we call add_numbers, it will automagically invoke the log_function_call decorator and the function's execution will be logged.

As seen, Python gives us the possibility to shorten up and make cleaner the use of the decorators through the use of the special symbol @.
In other languages, and before the 2.4 version of Python, we had to use the decorator by passing manually the function we wanted to decorate log_function_call(add_numbers(2,3)).

• Real life scenario

At this point you could ask:

"Ok, but how a decorator can be useful in a real context?"

You will be surprised but a very common scenario in which a decorator comes in handy is the authentication process.

Suppose we want to let a user access some resource only if authenticated. In a naive fashion, we could bind all the authentication checks inside the resource we want to protect, but because Python gives us a very convenient way to handle such a problem, we could instead use a decorator!

We can protect the resources by wrapping them in a decorator that applies the authentication logic for us, without the need to write it again for each resource we have or we want to add in the future. In this way, our source code will be cleaner, modular and make use of the language potentiality at full.

```
def authenticate(func):
    def wrapper(*args, **kwargs):
        # Check if the user is authenticated
        if check_authentication():
            # User is authenticated, execute the original function
            return func(*args, **kwargs)
        else:
            # User is not authenticated, return an error message
            return "Authentication failed. Please log in."
    return wrapper

@authenticate
def view_dashboard():
    return "Welcome to the dashboard!"

@authenticate
def view_profile():
    return "Here's your profile information."

# Example usage
print(view_dashboard())  # Output: "Welcome to the dashboard!"
print(view_profile())    # Output: "Authentication failed. Please log in."
```

As you can easily notice, we attached the decorator to two functions we wanted the authentication works on. If we followed a naive approach, the authentication logic (*hidden behind the made-up function* check_authentication ()) will have been duplicated inside each one of them making our source code more difficult to read and maintain.

2. Class Decorators

In some lines above, we said that a decorator can also be implemented as a **class**, but why use this approach over the function one?

Implementing decorators as classes can provide more flexibility and control compared to using functions. It allows decorators to maintain a state, accept additional arguments during initialization, and provide more complex behavior based on the instance attributes or methods.

To implement a decorator as a class we need to make use of one of the Python magic methods, the __call__() method.

__call__(): This method enables an instance of a class to be called as if it were a function. It allows objects to be callable and can be useful for implementing decorators or creating callable objects.

```
class DecoratorClass:
    def __init__(self, func):
        self.func = func

    def __call__(self, *args, **kwargs):
        # Code to be executed before the original function
        print("Decorator class: Before function execution")

        # Call the original function
        result = self.func(*args, **kwargs)

        # Code to be executed after the original function
        print("Decorator class: After function execution")

        return result

@DecoratorClass
def decorated_function():
    print("Original function")

# Using the decorated function
decorated_function()
```

• How it works

As you can notice, we defined a decorator class named DecoratorClass. The __init__() method initializes the class instance and stores the original function that will be decorated. The __call__()method is invoked when the decorated function is called. If before we had a inner function named “wrapper”, now its the magic method __call___ to serve as wrapper for the function we want to decorate, and in fact, as you can see, it adds some logging before and after the decorated function execution.

• **Real-life scenario — API Rate Limit**

Let's say you have a web API that provides various endpoints. We are being asked to implement the rate limit to certain endpoints in order to control the number of requests a client can make within a specific time period. In this case, a class decorator can be used to implement rate limiting behavior.

```
class RateLimit:
    def __init__(self, limit, interval):
        self.limit = limit
        self.interval = interval
        self.tokens = self.limit
        self.last_request_time = None

    def __call__(self, cls):
        original_handler = cls.handler

        def rate_limited_handler(*args, **kwargs):
            current_time = time.time()

            if self.last_request_time is None:
                self.tokens = self.limit
                self.last_request_time = current_time
            else:
                time_passed = current_time - self.last_request_time
                self.tokens += (time_passed / self.interval) * self.limit
                self.last_request_time = current_time

            if self.tokens >= 1:
                self.tokens -= 1
                return original_handler(*args, **kwargs)
            else:
                return "Rate limit exceeded"

        cls.handler = rate_limited_handler
        return cls

@RateLimit(limit=5, interval=60)
class APIEndpoint:
    def handler(self, request):
        # Process the request
        return "Request processed successfully"

# Usage
endpoint = APIEndpoint()
print(endpoint.handler("GET"))  # Output: Request processed successfully
print(endpoint.handler("GET"))  # Output: Request processed successfully
print(endpoint.handler("GET"))  # Output: Request processed successfully
print(endpoint.handler("GET"))  # Output: Request processed successfully
```

```
print(endpoint.handler("GET"))  # Output: Request processed successfully
print(endpoint.handler("GET"))  # Output: Rate limit exceeded
```

As you can easily notice we take advantage of the class decorator flexibility and defined two additional arguments to set up the rate limit.

More in general we can say that in real-life scenarios, class decorators can be useful for implementing cross-cutting concerns, such as logging, authentication, caching, or any behavior that needs to be applied consistently to multiple methods or classes.

Python Magic Methods

Python Magic methods *are the methods starting and ending with double underscores '__'. They are defined by built-in classes in Python and commonly used for operator overloading.*

They are also called **Dunder methods,** Dunder here means "Double Under (Underscores)".

Built in classes define many magic methods, **dir()** function can show you magic methods inherited by a class.

Example:

This code displays the magic methods inherited by **int** class.

Python

```
# code
print(dir(int))
```

Output

['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '__floor__', '__floordiv__', '__format__', '__ge__', '_...

Or, you can try cmd/terminal to get the list of magic functions in Python, open cmd or terminal, type python3 to go to the Python console, and type:

```
>>> dir(int)
```

Output:

```
__abs__                 __gt__                  __radd__                __setattr__
__add__                 __hash__                __rand__                __sizeof__
__and__                 __index__               __rdivmod__             __str__
__bool__                __init__                __reduce__              __sub__
__ceil__                __init_subclass__       __reduce_ex__           __subclasshook__
__class__               __int__                 __repr__                __truediv__
__delattr__             __invert__              __rfloordiv__           __trunc__
__dir__                 __le__                  __rlshift__             __xor__
__divmod__              __lshift__              __rmod__                as_integer_ratio
__doc__                 __lt__                  __rmul__                bit_count
__eq__                  __mod__                 __ror__                 bit_length
__float__               __mul__                 __round__               conjugate
__floor__               __ne__                  __rpow__                denominator
__floordiv__            __neg__                 __rrshift__             from_bytes
__format__              __new__                 __rshift__              imag
__ge__                  __or__                  __rsub__                numerator
__getattribute__        __pos__                 __rtruediv__            real
__getnewargs__          __pow__                 __rxor__                to_bytes
>>> |
```

Python Magic Methods
Below are the lists of Python magic methods and their uses.

Initialization and Construction

- ***__new__:*** *To get called in an object's instantiation.*
- ***__init__:*** *To get called by the __new__ method.*
- ***__del__:*** *It is the destructor.*

Numeric magic methods

- ***__trunc__(self):*** *Implements behavior for math.trunc()*
- ***__ceil__(self):*** *Implements behavior for math.ceil()*
- ***__floor__(self):*** *Implements behavior for math.floor()*
- ***__round__(self,n):*** *Implements behavior for the built-in round()*
- ***__invert__(self):*** *Implements behavior for inversion using the ~ operator.*
- ***__abs__(self):*** *Implements behavior for the built-in abs()*
- ***__neg__(self):*** *Implements behavior for negation*
- ***__pos__(self):*** *Implements behavior for unary positive*

Arithmetic operators

- *__add__(self, other): Implements behavior for the + operator (addition).*
- *__sub__(self, other): Implements behavior for the – operator (subtraction).*
- *__mul__(self, other): Implements behavior for the * operator (multiplication).*
- *__floordiv__(self, other): Implements behavior for the // operator (floor division).*
- *__truediv__(self, other): Implements behavior for the / operator (true division).*
- *__mod__(self, other): Implements behavior for the % operator (modulus).*
- *__divmod__(self, other): Implements behavior for the divmod() function.*
- *__pow__(self, other): Implements behavior for the ** operator (exponentiation).*
- *__lshift__(self, other): Implements behavior for the << operator (left bitwise shift).*
- *__rshift__(self, other): Implements behavior for the >> operator (right bitwise shift).*
- *__and__(self, other): Implements behavior for the & operator (bitwise and).*
- *__or__(self, other): Implements behavior for the | operator (bitwise or).*
- *__xor__(self, other): Implements behavior for the ^ operator (bitwise xor).*
- *__invert__(self): Implements behavior for bitwise NOT using the ~ operator.*

- *__neg__(self): Implements behavior for negation using the – operator.*
- *__pos__(self): Implements behavior for unary positive using the + operator.*

String Magic Methods

- ***__str__(self): Defines behavior for when str() is called on an instance of your class.***
- ***__repr__(self): To get called by built-int repr() method to return a machine readable representation of a type.***
- ***__unicode__(self): This method to return an unicode string of a type.***
- ***__format__(self, formatstr): return a new style of string.***
- ***__hash__(self): It has to return an integer, and its result is used for quick key comparison in dictionaries.***
- ***__nonzero__(self): Defines behavior for when bool() is called on an instance of your class.***
- ***__dir__(self): This method to return a list of attributes of a class.***
- ***__sizeof__(self): It return the size of the object.***

Comparison magic methods

- ***__eq__(self, other): Defines behavior for the equality operator, ==.***
- ***__ne__(self, other): Defines behavior for the inequality operator, !=.***
- ***__lt__(self, other): Defines behavior for the less-than operator, <.***
- ***__gt__(self, other): Defines behavior for the greater-than operator, >.***
- ***__le__(self, other): Defines behavior for the less-than-or-equal-to operator, <=.***
- ***__ge__(self, other): Defines behavior for the greater-than-or-equal-to operator, >=.***

Dunder or Magic Methods in Python

Let's see some of the Python magic methods with examples:

1. __init__ method

The __init__ method for initialization is invoked without any call, when an instance of a class is created, like constructors in certain other programming languages such as C++, Java, C#, PHP, etc.

These methods are the reason we can add two strings with the '+' operator without any explicit typecasting.

Python

```
# declare our own string class
class String:

    # magic method to initiate object
    def __init__(self, string):
        self.string = string

# Driver Code
if __name__ == '__main__':

    # object creation
    string1 = String('Hello')

    # print object location
    print(string1)
```

Output

<__main__.String object at 0x7f538c059050>

2. __repr__ method

__repr__method in Python defines how an object is presented as a string.

The below snippet of code prints only the memory address of the string object. Let's add a __repr__ method to represent our object.

Python

```
# declare our own string class
class String:

    # magic method to initiate object
    def __init__(self, string):
        self.string = string

    # print our string object
    def __repr__(self):
        return 'Object: {}'.format(self.string)

# Driver Code
if __name__ == '__main__':

    # object creation
    string1 = String('Hello')

    # print object location
    print(string1)
```

Output

Object: Hello

If we try to add a string to it:

```
# declare our own string class
class String:

    # magic method to initiate object
    def __init__(self, string):
        self.string = string

    # print our string object
    def __repr__(self):
        return 'Object: {}'.format(self.string)

# Driver Code
```

```
if __name__ == '__main__':

    # object creation
    string1 = String('Hello')

    # concatenate String object and a string
    print(string1 +' world')
```

Output:
TypeError: unsupported operand type(s) for +: 'String' and 'str'

3. __add__ method
__add__ method in Python defines how will be the objects of a class added together. It is also known as overloaded addition operator.

Now add __add__ method to String class :

Python

```
# declare our own string class
class String:

    # magic method to initiate object
    def __init__(self, string):
        self.string = string

    # print our string object
    def __repr__(self):
        return 'Object: {}'.format(self.string)

    def __add__(self, other):
        return self.string + other

# Driver Code
if __name__ == '__main__':

    # object creation
    string1 = String('Hello')

    # concatenate String object and a string
    print(string1 +' Geeks')
```

Output
Hello Geeks

Pseudocode:

 # Define a decorator for formatting

function bold_text(func):

 # Define the Report class

    # Class variable for storing templates

    # Constructor to initialize the report with title and content

    # Class method to add a template to the class variable

    # Class method to retrieve a template from the class variable

    # Magic method to call a report instance with a template name

    # String representation of the report

   # Define a simple template function

 # Define a fancy template function with bold formatting

 # Main function to generate and display reports

function main():

    # Add templates to the Report class

    # Create a report instance

    # Generate reports with different templates

    # Display the reports

 # Run the main function

if __name__ == "__main__":

    main()

Experiment 2 Title:

Experiment 2 Code:

Experiment 2 Output (Screenshot):

Bucket List Experiment Title:

Bucket List Experiment Code:

Bucket List Experiment Output (Screenshot):

Theory Questions (Handwritten):

Question 1: What is the purpose of a decorator in Python, and how can it be used to modify the behavior of a class method? Provide a brief example.

Questions 2: Explain what a generator is in Python and provide a basic example of how it differs from a regular function.

Conclusion:

The dynamic report generator demonstrates the use of decorators, class methods, and magic methods to create a flexible and customizable report generation system. This approach enables users to define and apply various formatting options dynamically, enhancing the reusability and maintainability of the code.